

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МОЭВМ

ОТЧЕТ
по лабораторной работе №4
по дисциплине «Вычислительная математика»

Студентка гр. 4343

Едина А.

Преподаватель

Лисс А.Р.

Санкт-Петербург

2026

Оглавление

Цель работы.....	3
Задание.....	4
Исследование функции.....	Error! Bookmark not defined.
Вывод.....	5
ПРИЛОЖЕНИЕ А.....	10
Исходный код программы.....	10

Цель работы

Реализовать численные методы интерполяции функции и исследовать их точность. Провести сравнительный анализ точности методов: интерполяционный многочлен Лагранжа и кубический сплайн.

Задание

Разработать программу для интерполяции функции $f(x) = \arctan\left(\frac{1}{x}\right) - x^2$ двумя методами:

1. Интерполяционный многочлен Лагранжа (с вычислением весов и нормировкой для устойчивости).
2. Закреплённый кубический сплайн с граничными условиями $S'(a) = f'(a), S'(b) = f'(b)$.

Для построения сплайна реализовать метод прогонки решения трёхдиагональной системы линейных уравнений.

Выполнение работы

Исследуемая функция: $f(x) = \arctan\left(\frac{1}{x}\right) - x^2$

Построим график функции.

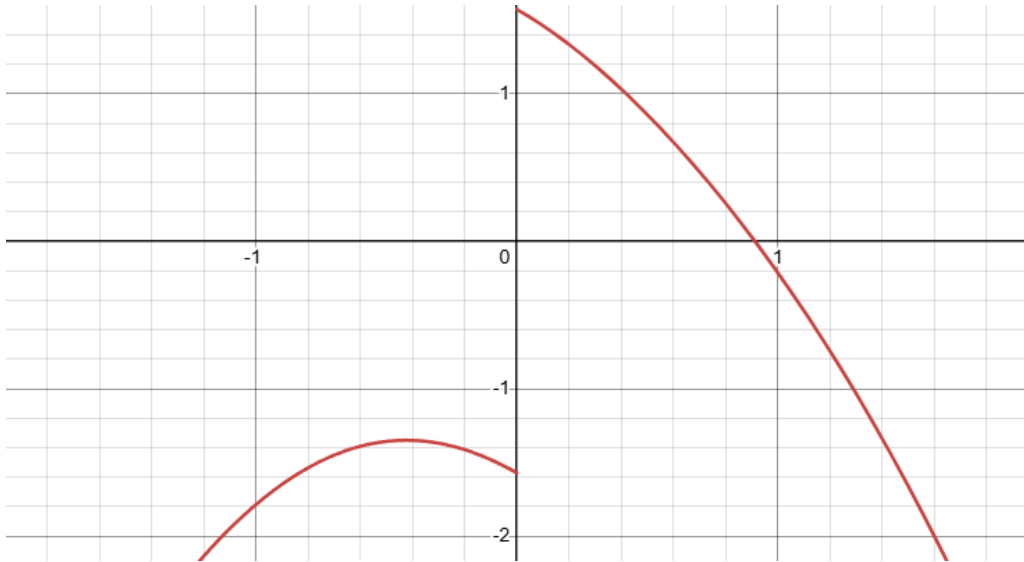


Рисунок 1 - график функции $f(x) = \arctan(1/x) - x^2$

Вычисление производной.

$$f'(x) = \frac{1}{1+\left(\frac{1}{x}\right)^2} * \left(-\frac{1}{x^2}\right) - 2x = -\frac{1}{x^2+1} - 2x$$

Программа написана на C++. Пользователь вводит количество узлов интерполяции n . Узлы выбираются равномерно на отрезке от 0.1 до 2. Для каждого узла вычисляется значение функции $f(x) = \arctg\left(\frac{1}{x}\right) - x^2$.

Для интерполяции Лагранжа используются барицентрические веса. Сначала для всех узлов вычисляются веса как обратные произведениям разностей координат. Затем веса нормируются для улучшения численной устойчивости. Значение многочлена в произвольной точке находится по формуле с весами.

Для построения закреплённого кубического сплайна составляется трёхдиагональная система уравнений относительно вторых производных в узлах. Граничные условия задаются через первую производную исходной функции. Система решается методом прогонки. После нахождения вторых

производных значение сплайна в любой точке вычисляется по формуле на каждом отрезке между узлами.

Затем на равномерной сетке из 1000 точек вычисляются значения исходной функции, многочлена Лагранжа и сплайна. Для каждой точки считаются абсолютные погрешности методов. Находятся максимальные погрешности на всей сетке.

Сравнительный анализ методов

Таблица 1 - сравнение максимальной ошибки на маленькой сетки

Кол-во узлов	Лагранж	Сплайн
20	0.0000000044	0.0000012670
40	0.0000001443	0.0000000691
60	0.0379089783	0.0000000131
80	99.3421285104	0.0000000041

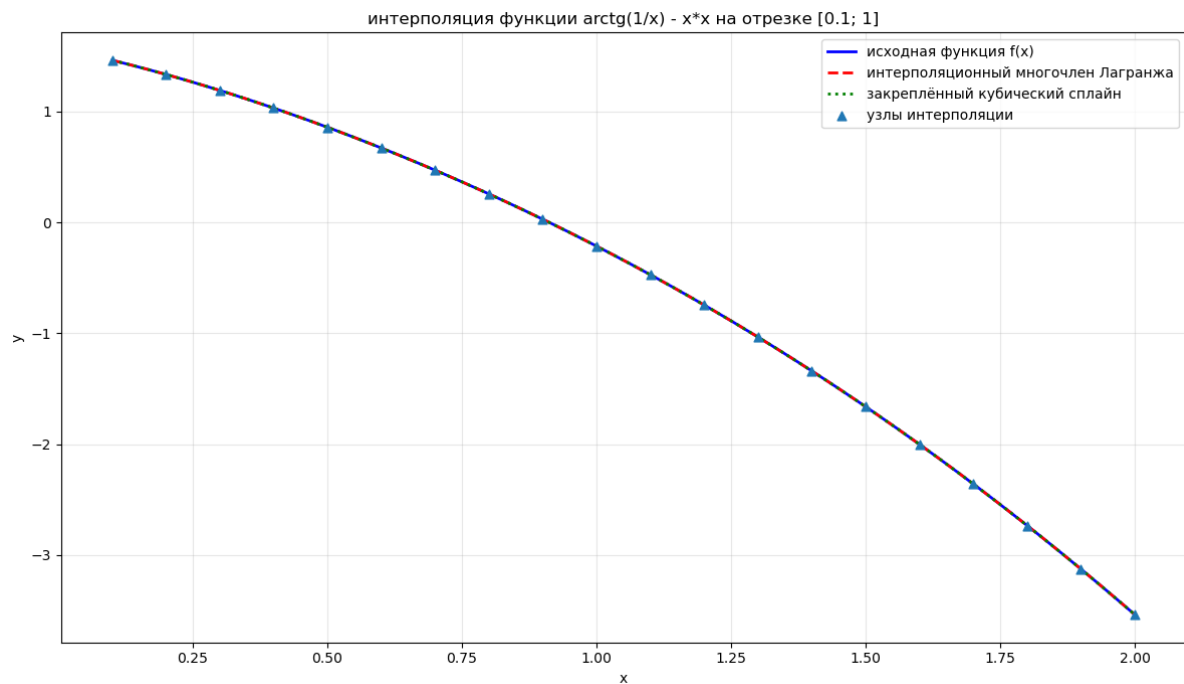


Рисунок 2 - интерполяция функция при $n = 20$

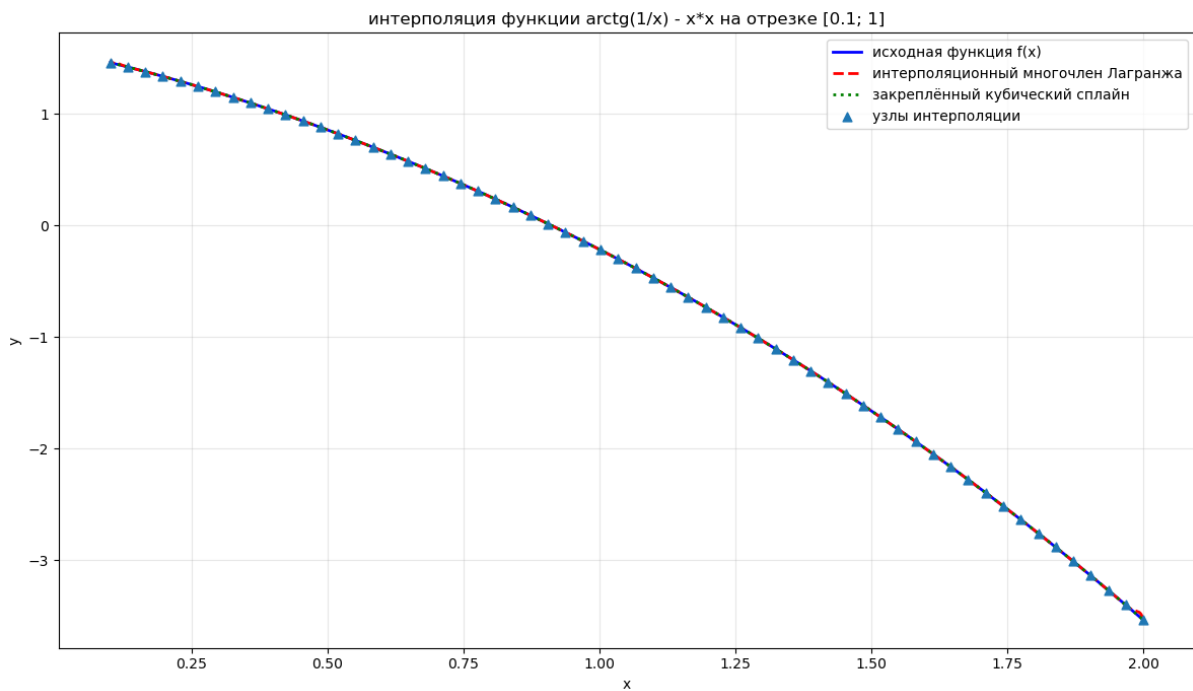


Рисунок 3 - интерполяция функции при $n = 60$

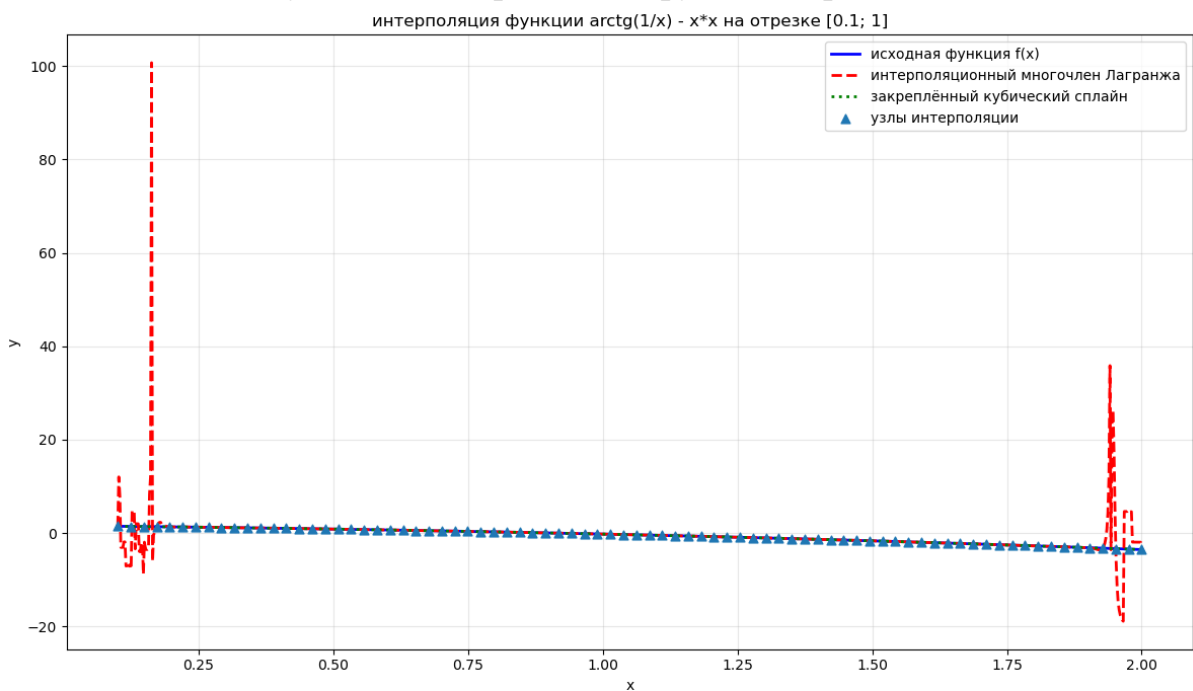


Рисунок 4 - интерполяция функции при $n = 80$

Только увеличив количество узлов до 80 становится наглядно видно недостаток метода полиномиальной интерполяции. График интерполяционного многочлена Лагранжа совершает резкие скачки вблизи границ, не соответствуя поведению исходной функции. Метод кубического сплайна демонстрирует качественную работу с минимальной ошибкой.

Дальнейшей увеличения количества узлов усугубляет ошибку для полинома Лагранжа.

ПРИЛОЖЕНИЕ А

Исходный код программы

```
#include <algorithm>
#include <cmath>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <limits>
#include <string>
#include <vector>

using namespace std;

double f(double x) {
    return atan(1.0 / x) - x*x;
}

double df(double x) {
    return - 1.0 / (1.0 + x * x) - 2*x;
}

// считая веса.  $w_i = 1 / \sum (x_i - x_j)$ 
// классика: от 0 до n-1 "+" ( $y_i * ( \sum (x - x_j) / \sum (x_i - x_j)$ 
// ) )
vector<double> barycentricWeights(const vector<double>& x) {
    const int nodeCount = (int)x.size();
    vector<double> weights(nodeCount, 1.0);

    for (size_t i = 0; i < nodeCount; ++i) {
        long double prod = 1.0L;
        for (size_t j = 0; j < nodeCount; ++j) {
            if (i == j) continue;
            prod *= (x[i] - x[j]);
        }
        weights[i] = 1.0 / (double)prod;
    }

    // нормирую веса ( $\sum < 1$  (улучшение численной устойчивости))
    double maxWeight = 0.0;
    for (double wi : weights) maxWeight = max(maxWeight, fabs(wi));
    if (maxWeight > 0) {
        for (double& wi : weights) wi /= maxWeight;
    }

    return weights;
}

double lagrangeBarycentric(const vector<double>& x, const
vector<double>& y,
                        const vector<double>& weights, double xq)
{
    const double eps = 1e-14;
    const int nodeCount = (int)x.size();
```

```

// проверка на совпадение с узлом
for (size_t i = 0; i < nodeCount; ++i) {
    if (fabs(xq - x[i]) < eps) return y[i];
}

long double up = 0.0L, down = 0.0L;
for (size_t i = 0; i < nodeCount; ++i) {
    long double part = (long double)weights[i] / (xq - x[i]);
    up += part * y[i];
    down += part;
}
return (double)(up / down);
}

// метод прогонки
// Решает трёхдиагональную систему:
//  $a[i] * x_{i-1} + b[i] * x_i + c[i] * x_{i+1} = d[i]$ 
vector<double> solveTridiagonal(const vector<double>& a, const
vector<double>& b,
                                const vector<double>& c, const
vector<double>& d) {
    const int n = (int)b.size();
    vector<double> cp(n, 0.0), dp(n, 0.0), x(n, 0.0);

    cp[0] = (n > 1 ? c[0] / b[0] : 0.0); // пограничные коэф.  $x_{i+1}$ 
    dp[0] = d[0] / b[0];

    //  $x_i = dp_i - (cp_i \cdot x_{i+1})$ 

    // прямой ход.
    for (size_t i = 1; i < n; ++i) {
        double divisor = b[i] - a[i] * cp[i - 1];
        if (i < n - 1) cp[i] = c[i] / divisor;
        dp[i] = (d[i] - a[i] * dp[i - 1]) / divisor;
    }

    // обратный. из последнего  $x_{n-1} = dp_{n-1}$ 
    x[n - 1] = dp[n - 1];
    for (int i = n - 2; i >= 0; --i) {
        x[i] = dp[i] - cp[i] * x[i + 1];
    }

    return x;
}

// натуральный кубический сплайн
//  $M[0] = M[n-1] = 0$ 
//  $M[i]$  - вторые производные в узлах
struct CubicSpline {
    vector<double> x;
    vector<double> y;
    vector<double> M; // вторые производные

```

```

double interpolate(double xq) const {
    const int n = (int)x.size();
    if (xq <= x.front()) return y.front();
    if (xq >= x.back()) return y.back();

    auto it = upper_bound(x.begin(), x.end(), xq);
    int i = (int)(it - x.begin()) - 1;

    double h = x[i + 1] - x[i];
    double A = (x[i + 1] - xq) / h;
    double B = (xq - x[i]) / h;

    double S =
        M[i] * pow(x[i + 1] - xq, 3) / (6.0 * h) +
        M[i + 1] * pow(xq - x[i], 3) / (6.0 * h) +
        (y[i] - M[i] * h * h / 6.0) * A +
        (y[i + 1] - M[i + 1] * h * h / 6.0) * B;

    return S;
}
};

// закреплённый кубический сплайн
// граничные условия:  $S'(a) = f'(a)$ ,  $S'(b) = f'(b)$ 
CubicSpline buildClampedCubicSpline(const vector<double>& x, const
vector<double>& y) {
    const int nodeCount = (int)x.size();
    CubicSpline spline;
    spline.x = x;
    spline.y = y;
    spline.M.assign(nodeCount, 0.0);

    if (nodeCount <= 2) return spline; // сплайн интерполяция не
определена

    const int N = nodeCount;
    vector<double> a(N, 0.0), b(N, 0.0), c(N, 0.0), d(N, 0.0);

    // левая граница ( $i = 0$ ).  $S'(x_0) = f'(x_0) \Rightarrow$ 
    double h1 = x[1] - x[0];
    b[0] = h1 / 3.0;
    c[0] = h1 / 6.0;
    d[0] = (y[1] - y[0]) / h1 - df(x[0]);

    // внутренние узлы ( $i = 1 \dots n-2$ )
    for (size_t i = 1; i <= nodeCount - 2; ++i) {
        double hPrev = x[i] - x[i - 1];
        double hNext = x[i + 1] - x[i];

        a[i] = hPrev / 6.0;
        b[i] = (hPrev + hNext) / 3.0;
        c[i] = hNext / 6.0;
        d[i] = (y[i + 1] - y[i]) / hNext - (y[i] - y[i - 1]) / hPrev;
    }

    // правая граница ( $i = n-1$ ).  $S'(x_{n-1}) = f'(x_{n-1})$ 

```

```

    double hLast = x[nodeCount - 1] - x[nodeCount - 2];
    a[nodeCount - 1] = hLast / 6.0;
    b[nodeCount - 1] = hLast / 3.0;
    d[nodeCount - 1] = df(x[nodeCount - 1]) - (y[nodeCount - 1] -
y[nodeCount - 2]) / hLast;

    // решаем трёхдиагональную систему
    vector<double> M_all = solveTridiagonal(a, b, c, d);

    for (size_t i = 0; i < nodeCount; ++i) {
        spline.M[i] = M_all[i];
    }

    return spline;
}

struct DataPoint {
    double x;
    double fx;
    double px;
    double sx;
    double errP;
    double errS;
};

int main() {
    const double A = 0.1;
    const double B = 2.0;

    int nodeCount;
    cout << "введите число узлов n: ";
    cin >> nodeCount;

    if (nodeCount < 3) {
        cerr << "ошибка: n должно быть >= 3\n";
        return 1;
    }

    vector<double> x(nodeCount), y(nodeCount);
    double h = (B - A) / (nodeCount - 1);

    for (size_t i = 0; i < nodeCount; ++i) {
        x[i] = A + i * h;
        y[i] = f(x[i]);
    }

    ofstream nodesOut("nodes.csv");
    nodesOut << "x,y\n";
    for (int i = 0; i < nodeCount; ++i) {
        nodesOut << setprecision(15) << x[i] << "," << y[i] << "\n";
    }
    nodesOut.close();

    // считаем узлы
    vector<double> w = barycentricWeights(x);

```

```

// сплайн
CubicSpline spline = buildClampedCubicSpline(x, y);

// выводим узлы
cout << fixed << setprecision(10);
cout << "\узлы интерполяции:\n";
cout << "i\t x_i\t\t f(x_i)\n";
for (size_t i = 0; i < nodeCount; ++i) {
    cout << i << "\t " << x[i] << "\t " << y[i] << "\n";
}

// сетка
const int Nfine = 1000;
vector<DataPoint> datas;
datas.reserve(Nfine + 1);

double maxErrP = 0.0;
double maxErrS = 0.0;

for (int i = 0; i <= Nfine; ++i) {
    double t = (double)i / Nfine;
    double xx = A + (B - A) * t;

    double fx = f(xx);
    double px = lagrangeBarycentric(x, y, w, xx);
    double sx = spline.interpolate(xx);

    double errP = fabs(px - fx);
    double errS = fabs(sx - fx);

    maxErrP = max(maxErrP, errP);
    maxErrS = max(maxErrS, errS);

    datas.push_back({xx, fx, px, sx, errP, errS});
}

cout << "\максимальная ошибка на мелкой сетке:\n";
cout << "Lagrange = " << maxErrP << "\n";
cout << "Spline   = " << maxErrS << "\n";

ofstream out("data.csv");
out << "x,f(x),lagrange,spline,err_lagrange,err_spline\n";
for (const auto& s : datas) {
    out << setprecision(15)
        << s.x << ", "
        << s.fx << ", "
        << s.px << ", "
        << s.sx << ", "
        << s.errP << ", "
        << s.errS << "\n";
}
out.close();

cout << "\данные сохранены в файл data.csv\n";

return 0;

```

}